

Mock Exchange

Real-Time Trading Platform

Project Specification Document

Project	Mock Exchange - Real-Time Trading Platform
Version	0.1.0 (Phase 1 Complete)
Architecture	Microservices, Event Sourcing, CQRS
Tech Stack	NestJS / PostgreSQL / Redis / Kafka
Date	2026-02-17

Table of Contents

- 1. Project Overview
- 2. System Architecture
- 3. Service Details
- 4. Event Sourcing & CQRS
- 5. Order Placement Flow
- 6. Database Schema
- 7. API Specification
- 8. Technology Distribution
- 9. Price Simulation Engine
- 10. Development Roadmap

1. Project Overview

Mock Exchange is a production-grade, real-time mock stock and crypto trading platform designed as a comprehensive microservices architecture showcase. The system simulates realistic market conditions using Geometric Brownian Motion (GBM) for price generation, supports market and limit orders through an event-sourced order engine, and manages user portfolios with precise decimal arithmetic for financial calculations.

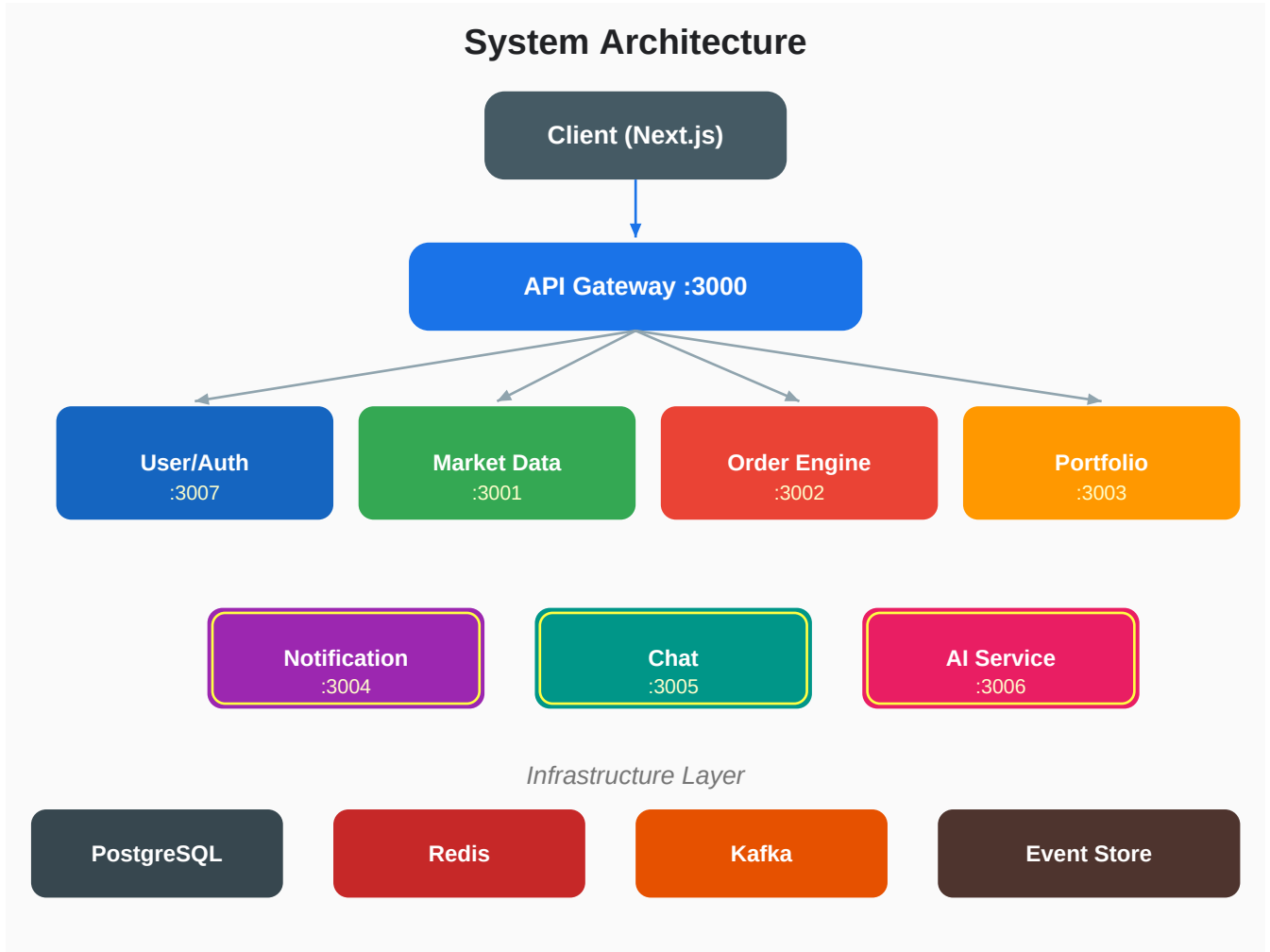
The platform is built to handle 1,000+ concurrent traders with horizontal scalability, featuring real-time price streaming via WebSocket, asynchronous event processing through Kafka, and a CQRS pattern that separates write and read models for optimal performance.

Key Features

Feature	Description	Status
User Authentication	JWT + Refresh Token rotation	Complete
Market Data	GBM price simulation, 10 assets	Complete
Order Engine	Event-sourced, market/limit orders	Complete
Portfolio Mgmt	Balance, holdings, settlement	Complete
API Gateway	JWT validation, rate limiting, proxy	Complete
Real-time Prices	WebSocket streaming via Socket.IO	Planned
Chat System	Real-time messaging between traders	Planned
AI Analysis	Trading signals and portfolio insights	Planned

2. System Architecture

The system follows a microservices architecture with API Gateway pattern. Each service is independently deployable, owns its database (Database-per-Service), and communicates through HTTP for synchronous calls and Kafka for asynchronous event processing.



Architecture Principles

- **Single Responsibility:** Each service handles one bounded context
- **Database-per-Service:** Independent PostgreSQL databases prevent coupling
- **Event-Driven:** Kafka enables async communication and eventual consistency
- **API Gateway:** Centralized authentication, rate limiting, and routing
- **CQRS:** Separate write (Event Store) and read (PostgreSQL) models
- **Idempotency:** All write operations support idempotency keys

3. Service Details

API Gateway (:3000)

Entry point for all client requests. Validates JWT tokens, applies rate limiting (100 req/min), and proxies requests to downstream services. Includes WebSocket gateway for real-time price streaming.

Components: JWT Authentication • Rate Limiting (Throttler) • HTTP Proxy • WebSocket Gateway

User/Auth Service (:3007)

Handles user registration, authentication, and token management. Uses bcrypt for password hashing and SHA-256 for refresh token hashing. Implements token rotation for security.

Components: Register/Login • JWT Access Token (15m) • Refresh Token Rotation (7d) • httpOnly Cookie

Market Data Service (:3001)

Simulates realistic price movements using Geometric Brownian Motion. Supports 10 assets (BTC, ETH, SOL, XRP, DOGE, AAPL, GOOGL, TSLA, MSFT, NVDA) with configurable volatility and drift.

Components: GBM Price Engine • Redis Price Cache • Kafka Event Publishing • Candlestick Aggregation

Order Engine (:3002)

Event-sourced order management with CQRS pattern. Supports market and limit orders with an in-memory matching engine using price-time priority.

Components: Event Sourcing (AggregateRoot) • In-Memory Order Book • CQRS Read Model Projection • Idempotency Check

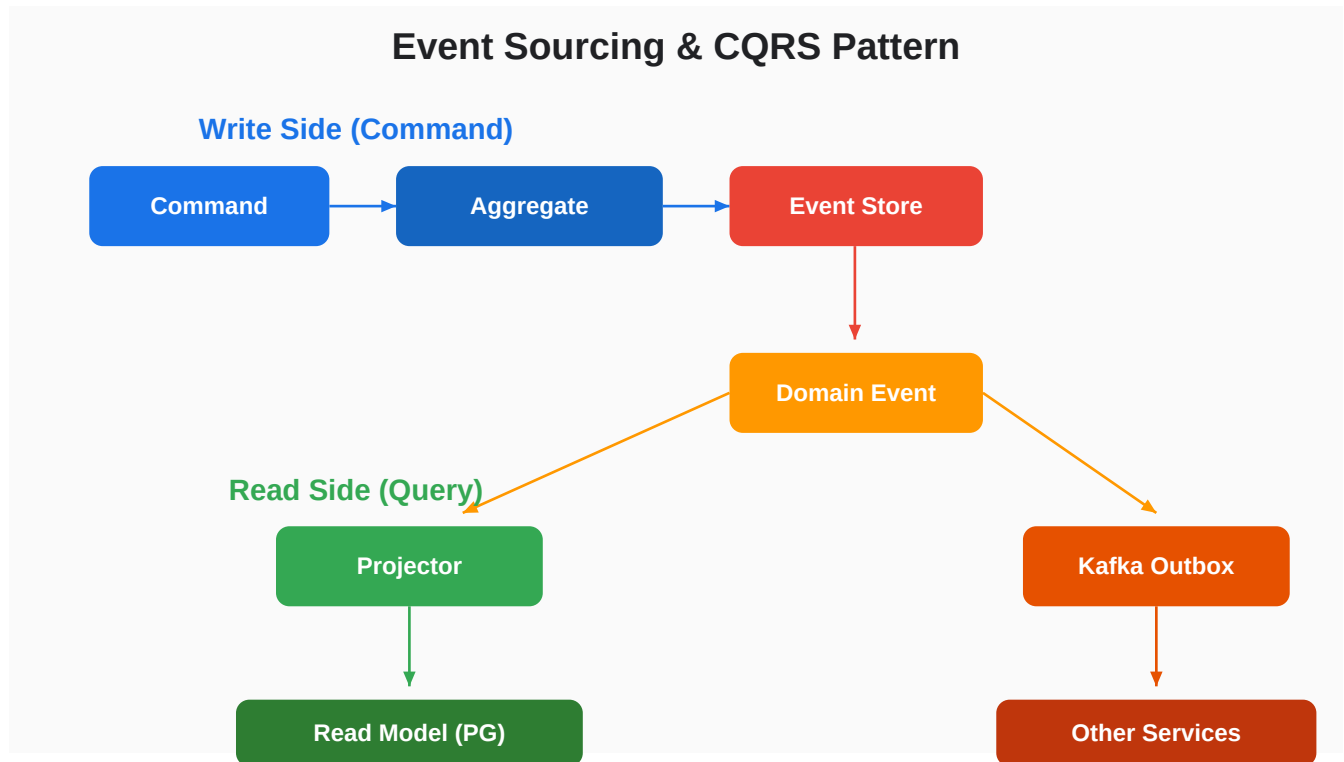
Portfolio Service (:3003)

Manages user balances, holdings, and trade settlement. Uses Decimal.js for precise financial calculations and weighted average cost basis tracking.

Components: Deposit/Balance Management • Fund Reservation (Order Flow) • Buy/Sell Settlement • Weighted Avg Cost Basis

4. Event Sourcing & CQRS

The Order Engine uses Event Sourcing to persist all state changes as immutable events. Instead of storing the current state, the system records every event that occurs (OrderPlaced, OrderMatched, OrderFilled, OrderCancelled). The current state is reconstructed by replaying events from the Event Store.

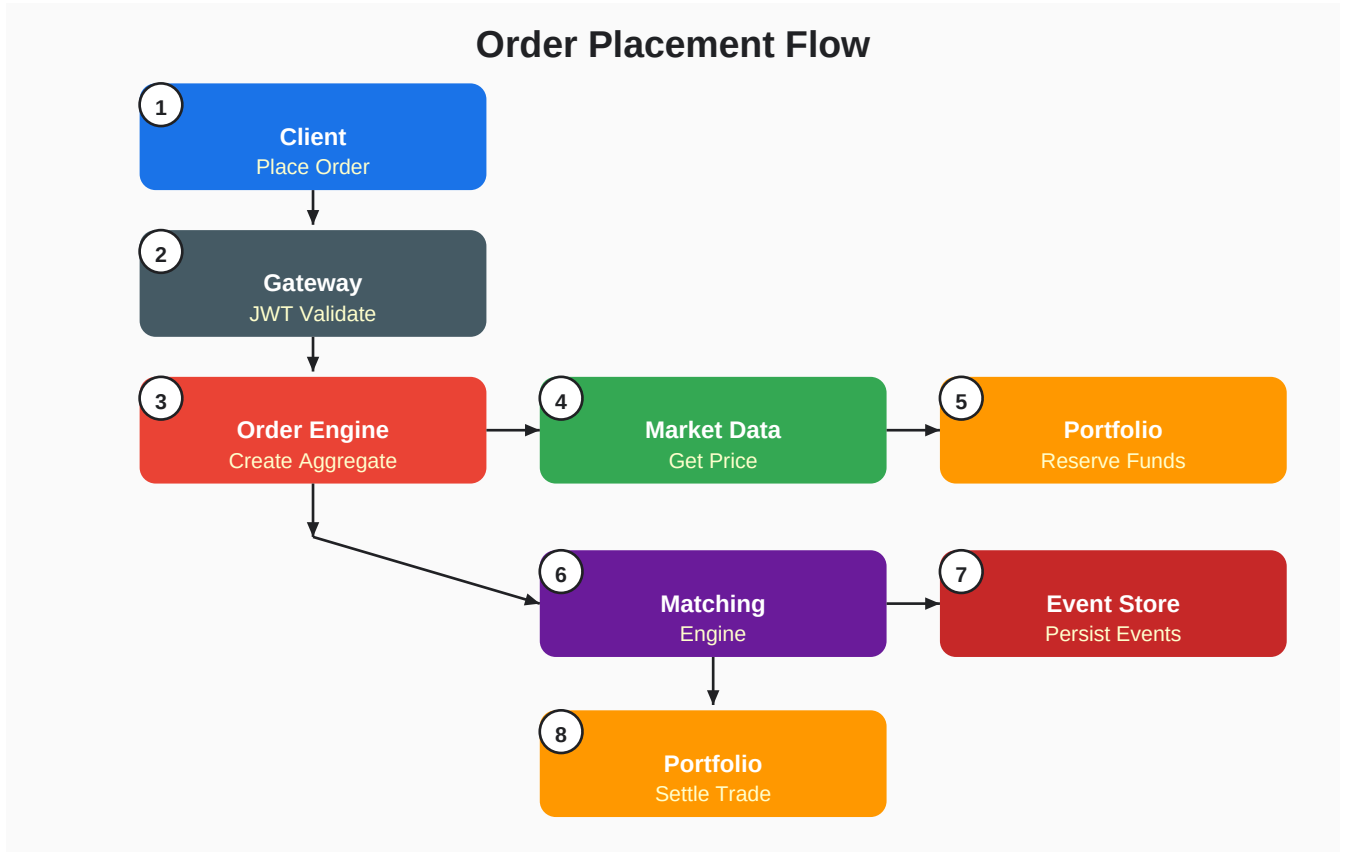


Event Types

Event Type	Trigger	Data
OrderPlaced	User submits order	orderId, symbol, side, type, quantity, price
OrderMatched	Matching engine finds match	tradeId, matchedQty, matchedPrice
OrderFilled	Order fully executed	totalFilledQty, averagePrice
OrderCancelled	User or system cancels	reason, unfilledQuantity

5. Order Placement Flow

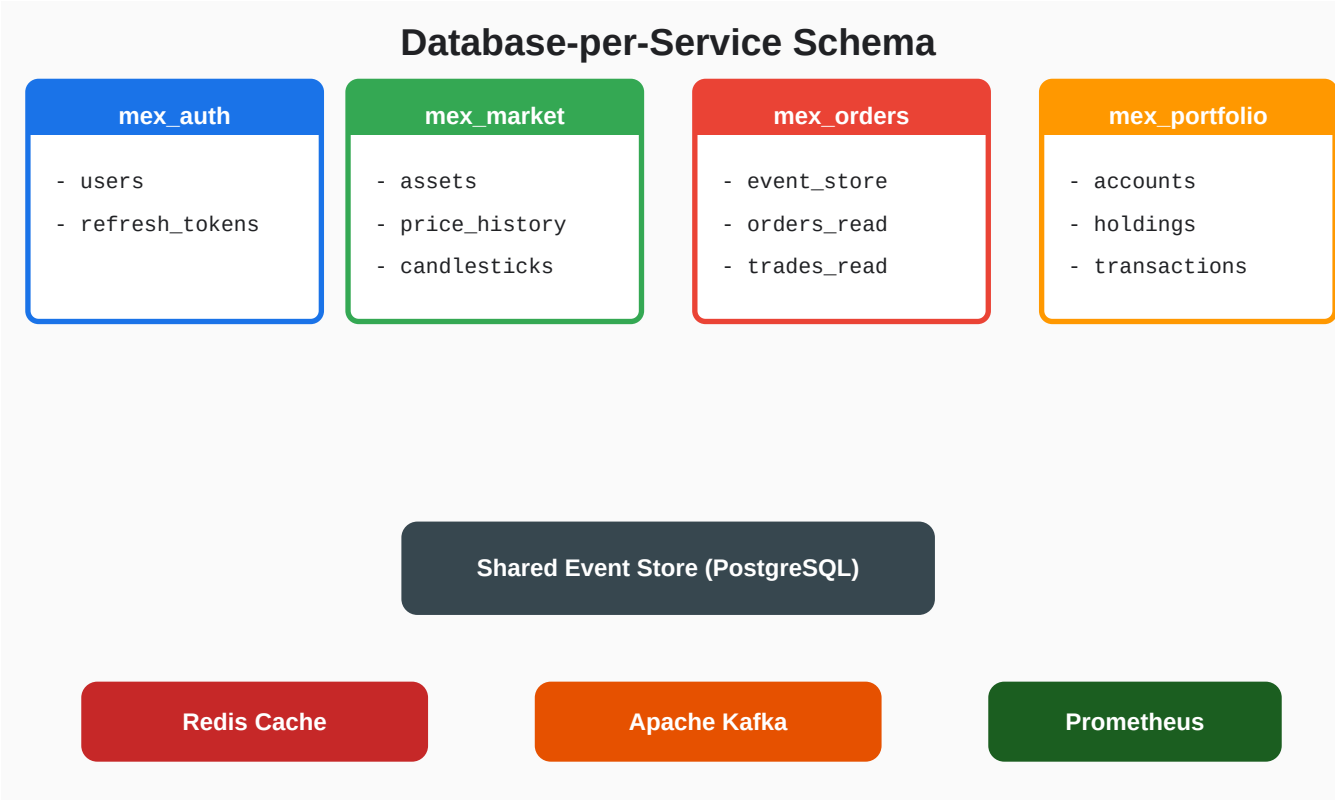
When a user places an order, the system orchestrates multiple services in a saga-like pattern: validating the request, checking market prices, reserving funds, executing the matching engine, persisting events, and settling trades.



1. Client sends POST /api/orders with JWT token
2. API Gateway validates JWT, extracts userId, proxies to Order Engine
3. Order Engine checks idempotency key against read model
4. Fetches current market price from Market Data Service
5. Reserves funds in Portfolio Service (for BUY orders)
6. Creates OrderAggregate, raises ORDER_PLACED event
7. Persists event to Event Store with Outbox entry
8. For MARKET orders: Matching Engine executes immediately
9. Raises ORDER_MATCHED / ORDER_FILLED events
10. Settles trade in Portfolio Service (buy/sell settlement)

6. Database Schema

Each service owns its database following the Database-per-Service pattern. All monetary values use Decimal(20,8) for precision. The Event Store uses a shared PostgreSQL database with optimistic concurrency control.



Database	Tables	Purpose
mex_auth	users, refresh_tokens	User accounts and auth tokens
mex_market	assets, price_history, candlesticks	Market data and price feeds
mex_orders	event_store, orders_read, trades_read	Order events and CQRS projections
mex_portfolio	accounts, holdings, transactions	Balance and position tracking

7. API Specification

Authentication API (/api/auth)

Method	Path	Auth	Description
POST	/register	No	Register new user
POST	/login	No	Login, returns JWT + refresh cookie
POST	/refresh	Cookie	Refresh access token
POST	/logout	Cookie	Revoke tokens
GET	/me	JWT	Get current user profile

Market Data API (/api/market)

Method	Path	Auth	Description
GET	/assets	No	List all tradable assets
GET	/prices	No	Latest prices for all assets
GET	/prices/:symbol	No	Price for specific symbol
GET	/prices/:symbol/history	No	Historical price data
GET	/prices/:symbol/candlesticks	No	OHLCV candlestick data

Order API (/api/orders)

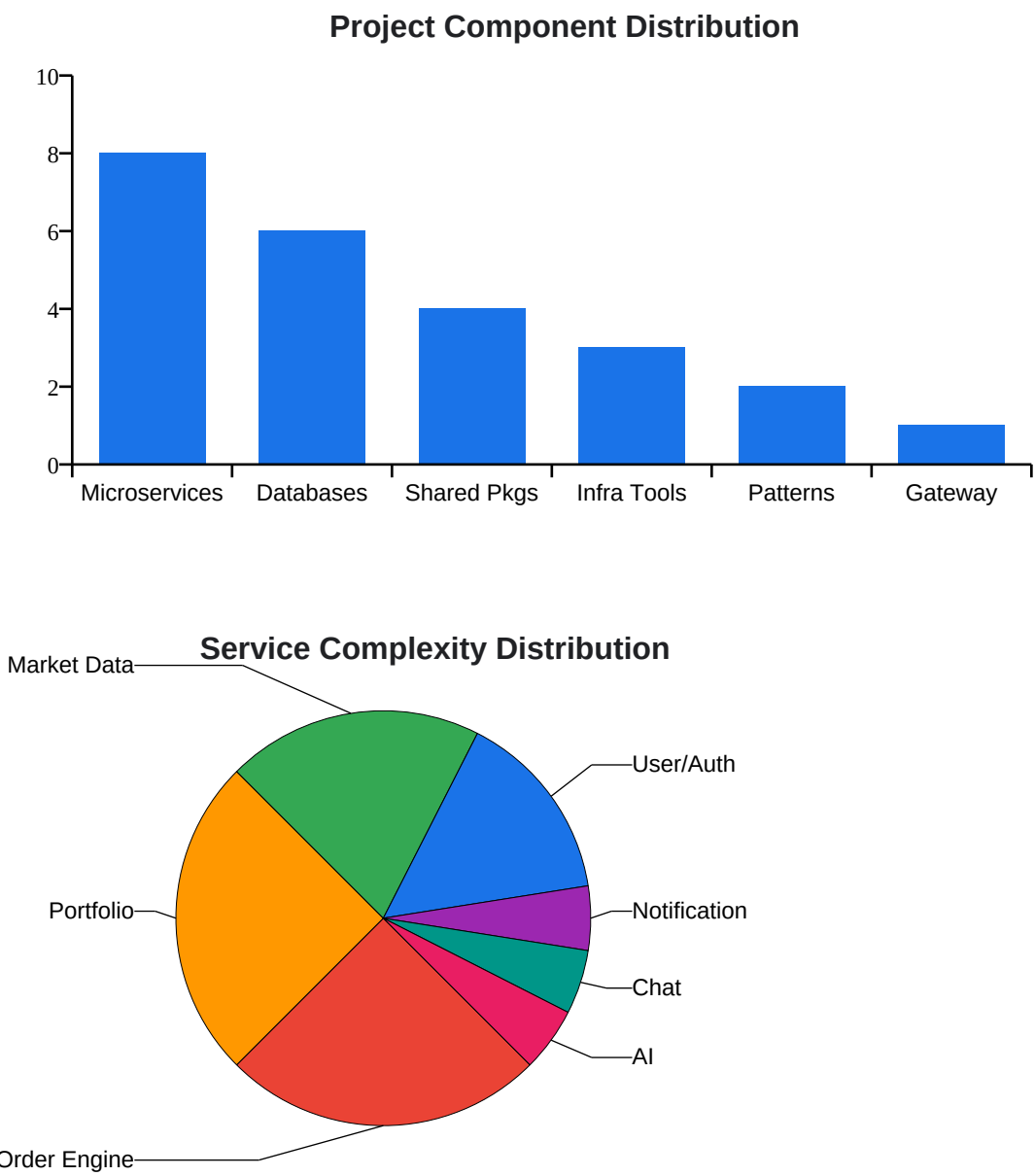
Method	Path	Auth	Description
POST	/	JWT	Place new order (market/limit)
GET	/	JWT	List user orders (paginated)
GET	/:orderId	JWT	Get order detail
DELETE	/:orderId	JWT	Cancel pending order
GET	/trades/history	JWT	User trade history
GET	/book/:symbol	No	Order book depth

Portfolio API (/api/portfolio)

Method	Path	Auth	Description
POST	/deposit	JWT	Deposit funds
GET	/balance	JWT	Cash balance
GET	/holdings	JWT	Asset holdings
GET	/summary	JWT	Full portfolio summary
GET	/transactions	JWT	Transaction history

8. Technology Distribution

The project is organized into 10 buildable packages: 2 shared libraries and 8 microservices. The chart below shows the distribution of components across different categories.

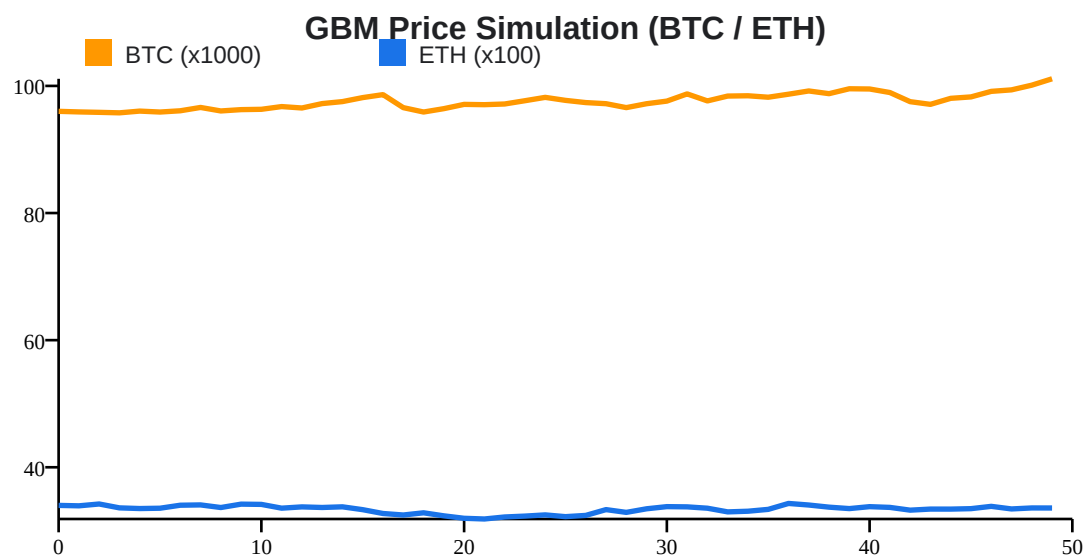


9. Price Simulation Engine

The Market Data Service uses Geometric Brownian Motion (GBM) to simulate realistic price movements. The model follows the stochastic differential equation:

$$dS = \mu * S * dt + \sigma * S * dw$$

Where S is the asset price, mu is the drift (expected return), sigma is the volatility, dt is the time step, and dW is a Wiener process (random walk). Each asset has configurable base price, drift, and volatility parameters.



Symbol	Type	Base Price	Volatility	Drift
BTC	Crypto	\$96,000	2.5%	0.02%
ETH	Crypto	\$3,400	3.0%	0.01%
SOL	Crypto	\$190	4.0%	0.03%
AAPL	Stock	\$185	1.2%	0.005%
TSLA	Stock	\$380	3.5%	0.01%
NVDA	Stock	\$900	3.0%	0.02%

10. Development Roadmap

[DONE] Phase 0: Foundation

- Monorepo setup (Turborepo + pnpm)
- Docker Compose (PostgreSQL, Redis, Kafka)
- Shared packages (common, event-store)
- User/Auth Service with JWT
- API Gateway with rate limiting
- CI/CD Pipeline (GitHub Actions)

[DONE] Phase 1: Core Trading MVP

- Market Data Service with GBM price engine
- Order Engine with Event Sourcing + CQRS
- Portfolio Service with balance management
- API Gateway proxy wiring for all services
- Trade settlement flow

[TODO] Phase 2: Advanced Trading

- Enhanced limit order matching
- Candlestick charts and technical indicators
- P&L; calculation and portfolio analytics
- WebSocket real-time price streaming

[TODO] Phase 3: Social Features

- Real-time chat between traders
- Push notifications (email, WebSocket)
- Activity feed and social features

[TODO] Phase 4: AI Integration

- Trading signal generation

- Portfolio analysis and recommendations
- Market sentiment analysis

[TODO] Phase 5: Production

- Kubernetes deployment manifests
- Helm charts and service mesh
- Observability (Prometheus, Grafana, Jaeger)
- Load testing and performance optimization